

Université Versailles Saint-Quentin-en-Yvelines

Master CHPS

TP1 AOC

Optimisation et analyse de performance avec MAQAO

Réalisé par :

Arab Lina

Année universitaire : 2025/2026

Date : Janvier 2026

Table des matières

1	Contexte et objectif	2
2	Configuration matérielle et environnement	2
3	Méthodologie de mesure et outils	2
3.1	Outils	2
3.2	Commandes exécutées	3
3.3	Principe de lecture des performances	3
4	Analyse de la version initiale : goulots d'étranglement	3
4.1	Ce que fait le code initial	3
4.2	Pourquoi c'est lent	3
5	Optimisations incrémentales et impact sur le temps total	3
5.1	Analyse étape par étape	4
5.2	Accélération globale	4
6	Résultats MAQAO	4
6.1	Version initiale	4
6.2	Version optimisée	5
7	Discussion : pourquoi ces optimisations marchent	5
7.1	IPC, stalls et hiérarchie mémoire	5
7.2	Conditions pratiques pour la vectorisation	6
7.3	Impact de <code>exp</code>	6
8	Preuve de cohérence des résultats	6
9	Limites et pistes d'amélioration	6
10	Conclusion	7

Résumé

Dans ce TP, j’optimise un code C qui simule l’évolution temporelle d’un champ complexe 2D sur une grille $N \times N$ avec un stencil à 5 points, puis je calcule l’énergie à chaque pas de temps. Je pars d’une version non optimisée avec I/O ASCII, organisation AoS et double parcours mémoire, puis j’applique des optimisations ciblées comme SoA, suppression de l’I/O, fusion de boucles, accès stride-1, alignement, **restrict** et SIMD [1, 2]. Avec $N = 5000$ et $steps = 10$, je passe d’environ 206 s à 6.77 s, soit un speedup d’environ $30\times$.

1 Contexte et objectif

Le programme met à jour un champ complexe $u(i, j) = u_{re}(i, j) + iu_{im}(i, j)$ sur une grille 2D. À chaque pas de temps, la mise à jour suit :

$$u^{t+1} = u^t + \alpha \Delta u^t + \beta e^{-\gamma |u^t|^2} u^t,$$

où Δu est le Laplacien discret (stencil 5 points) :

$$\Delta u(i, j) = u(i-1, j) + u(i+1, j) + u(i, j-1) + u(i, j+1) - 4u(i, j).$$

L’énergie calculée à chaque pas de temps est :

$$E = \sum_{i=1}^{N-2} \sum_{j=1}^{N-2} (u_{re}(i, j)^2 + u_{im}(i, j)^2).$$

Mon objectif est de réduire le temps d’exécution total, puis d’expliquer les gains observés avec des notions liées à l’IPC, aux stalls mémoire, à la hiérarchie cache et à l’exploitation de SIMD [1, 2].

2 Configuration matérielle et environnement

Les tests ont été réalisés sous WSL2 (Hyper-V) avec la machine suivante (commande `lscpu`) :

CPU	Intel Core i5-8350U @ 1.70GHz
Cœurs / Threads	4 cœurs, 8 threads
Micro-architecture	Kaby Lake
SIMD	AVX, AVX2, FMA
Caches	L1d 128 KiB (4), L1i 128 KiB (4), L2 1 MiB (4), L3 6 MiB
Environnement	WSL2 (Microsoft Hypervisor)

TABLE 1 – Caractéristiques principales de la machine utilisée.

3 Méthodologie de mesure et outils

3.1 Outils

J’utilise MAQAO OneView (rapport HTML) pour analyser les performances et la vectorisation. Pour autoriser l’accès aux compteurs matériels :

```
sudo sysctl -w kernel.perf_event_paranoid=1
```

J’installe aussi :

```
sudo apt install hwloc-nox
```

3.2 Commandes exécutées

Version initiale (baseline)

```
gcc -O0 -g -fno-omit-frame-pointer -o app_00 student_cx.c -lm
./maqao oneview -R1 ./app_00 -- 5000 10
```

Version optimisée (finale)

```
gcc -O3 -march=native -g -fno-omit-frame-pointer -fopenmp \
-o app_opt_mem optimized_tp_mem.c -lm
./maqao oneview -R1 ./app_opt_mem -- 5000 10
```

3.3 Principe de lecture des performances

Pour interpréter mes mesures, j'utilise le modèle :

$$T = \frac{IC \times CPI}{f},$$

où IC est le nombre d'instructions, CPI les cycles par instruction, et f la fréquence [1]. Réduire le temps d'exécution peut se faire en diminuant IC ou en diminuant CPI [1]. Dans mon cas, certaines optimisations réduisent surtout IC (suppression I/O ASCII), et d'autres réduisent surtout CPI en diminuant les stalls et en exploitant SIMD [2].

4 Analyse de la version initiale : goulots d'étranglement

4.1 Ce que fait le code initial

Dans la version initiale :

- je génère un fichier ASCII contenant N^2 nombres complexes avec `fprintf`,
- je relis ensuite le fichier avec `fscanf`,
- à chaque pas de temps, je calcule la mise à jour du champ, puis je fais une seconde boucle complète pour calculer l'énergie.

4.2 Pourquoi c'est lent

I/O ASCII. Écrire et relire des millions de valeurs en ASCII implique beaucoup d'appels système et de conversions texte vers flottants. Cette partie masque le temps de calcul réel et augmente fortement le nombre d'instructions exécutées.

Organisation AoS. Le champ est stocké en *Array of Structures* (AoS) avec `cplx{re,im}`. Les composantes réelle et imaginaire sont entremêlées en mémoire, ce qui rend l'accès contigu à une seule composante moins direct et peut compliquer l'utilisation efficace de SIMD [2].

Deux parcours mémoire par pas de temps. Après la mise à jour, la fonction `compute_energy` relit tout `unew`. Cela augmente le trafic mémoire et peut augmenter le CPI à cause des attentes liées aux accès mémoire [1].

Stencil et coût de exp. Le stencil utilise plusieurs lectures voisines par cellule, et `exp()` est une opération coûteuse. Même si une partie de la boucle se vectorise, l'appel à une fonction mathématique lourde peut limiter le gain total [2].

5 Optimisations incrémentales et impact sur le temps total

Pour mesurer l'effet de chaque optimisation, j'ai construit des versions intermédiaires. À chaque étape, j'ajoute une seule optimisation. Toutes les mesures utilisent $N = 5000$ et $steps = 10$.

Version	Optimisation ajoutée (cumulative)	Temps total (s)
Avant	Code initial (baseline)	206.00
optim1.c	Passage AoS → SoA (I/O ASCII conservée)	200.00
optim2.c	Suppression I/O ASCII, génération en mémoire	30.76
optim3.c	Fusion du calcul d'énergie	29.28
optim4.c	Accès stride-1 et indexation simplifiée	29.18
optim5.c	Alignement mémoire	10.05
optim6.c	<code>restrict</code> et SIMD	6.77

TABLE 2 – Temps total après chaque optimisation incrémentale.

5.1 Analyse étape par étape

optim1 : AoS vers SoA. Je stocke séparément `re[]` et `im[]`. Ce format rend les accès unit-stride sur chaque composante plus simples et facilite la vectorisation [2]. Le gain reste faible car l'I/O ASCII domine encore.

optim2 : suppression de l'I/O ASCII. Je génère directement les données en mémoire et je supprime `fprintf/fscanf`. Le nombre total d'instructions et le coût système diminuent fortement, ce qui explique le gain principal.

optim3 : fusion du calcul d'énergie. Au lieu de faire une deuxième passe complète, je calcule E dans la même boucle que la mise à jour. Je réduis le trafic mémoire et je limite des relectures inutiles.

optim4 : accès stride-1 et indexation simplifiée. En C, l'ordre mémoire est row-major, donc les accès contigus correspondent à la variation de j . Je mets j en boucle interne et je calcule `id=row+j`. Cela améliore la localité et favorise le préchargement matériel et SIMD [2].

optim5 : alignement mémoire. J'alloue les tableaux avec `posix_memalign` et j'indique l'alignement via `__builtin_assume_aligned`. L'alignement réduit les pénalités sur les accès vectoriels et simplifie la génération d'instructions SIMD [2]. Le gain observé est important.

optim6 : restrict et SIMD. En C, l'aliasing potentiel entre pointeurs peut empêcher la vectorisation. Le mot-clé `restrict` indique que `ure`, `uim`, `unewre` et `unewim` ne se recouvrent pas, ce qui permet au compilateur d'optimiser plus agressivement [2]. J'ajoute aussi `#pragma omp simd reduction(+:E)` pour encourager la vectorisation et gérer la réduction.

5.2 Accélération globale

$$Speedup = \frac{206.00}{6.77} \approx 30.4$$

La version finale est donc environ $30\times$ plus rapide que la version initiale.

6 Résultats MAQAO

6.1 Version initiale

Observations :

- Temps total proche de 206 s.
- Efficacité d'accès mémoire autour de 62%.
- Score de compilation faible, cohérent avec `-O0`.

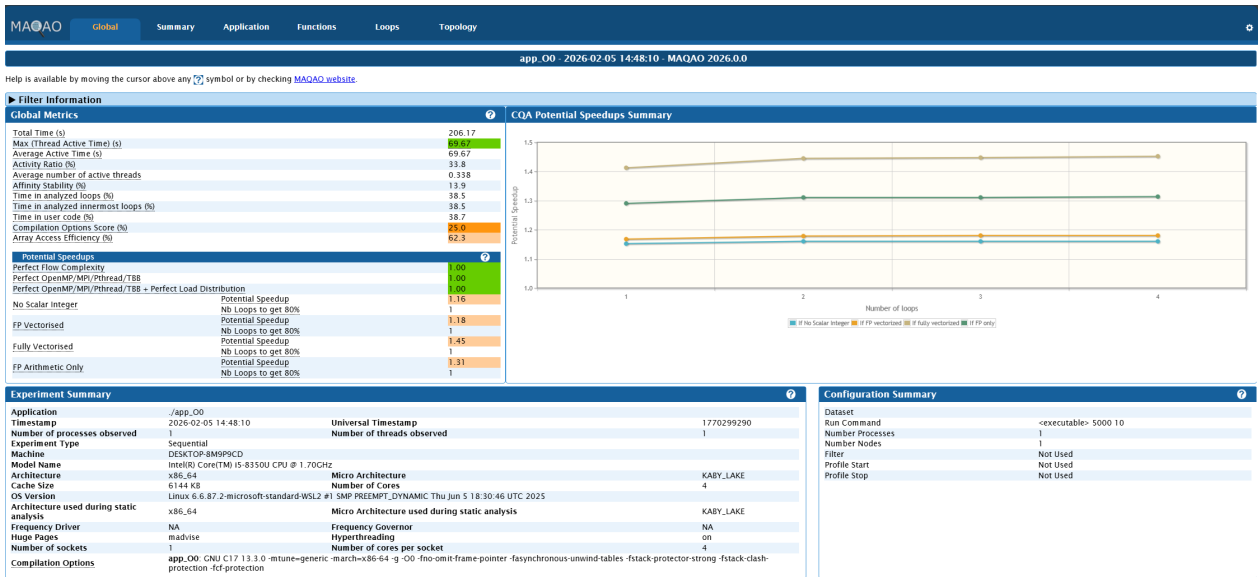


FIGURE 1 – Rapport MAQAO OneView pour la version initiale.

6.2 Version optimisée

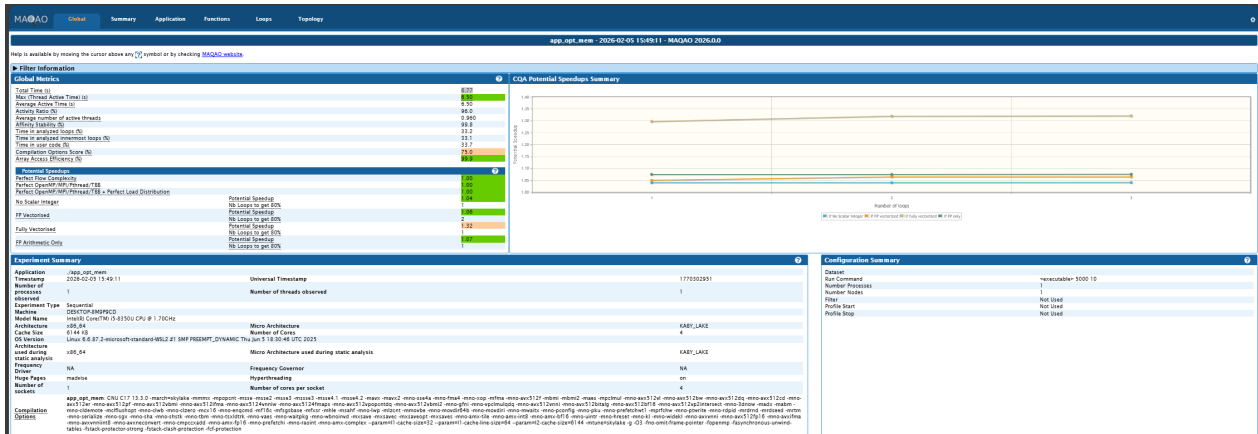


FIGURE 2 – Rapport MAQAO OneView pour la version optimisée.

Observations :

- Temps total proche de 6.77s.
- Efficacité d'accès mémoire proche de 100%.
- Score de compilation élevé, cohérent avec -O3 -march=native.

7 Discussion : pourquoi ces optimisations marchent

7.1 IPC, stalls et hiérarchie mémoire

Le débit d'exécution dépend de l'IPC. Quand le processeur attend des données, le pipeline insère des stalls et le *CPI* augmente [1]. En améliorant la localité (stride-1), en réduisant le trafic (fusion de boucles) et en alignant les données, je réduis les attentes mémoire et j'augmente l'IPC effectif [1]. La vectorisation SIMD augmente aussi la quantité de travail par instruction, ce qui améliore le débit global [2].

7.2 Conditions pratiques pour la vectorisation

Une vectorisation efficace nécessite une boucle régulière, une indépendance entre itérations, un aliasing contrôlé et des accès contigus [2]. Dans la version finale, j'ai SoA, `restrict`, stride-1 et alignement, ce qui rend la boucle interne plus facile à vectoriser.

7.3 Impact de exp

Le stencil se prête bien à SIMD, mais `exp` peut rester coûteuse. Le gain total peut donc être inférieur au facteur théorique lié à la largeur SIMD, car une partie du temps est prise par une fonction mathématique lourde [2].

8 Preuve de cohérence des résultats

Je compare les sorties (énergie à chaque pas) entre versions.

```
linaarab0@DESKTOP-8M9P9CD:/mnt/c/Users/Microsoft/Desktop/UVSQ/AOC/TP1$ ./app_opt_mem 5000 10
Generating random input in memory
Computing 10 time steps
step 0 energy = 1.3813470141e+07
step 1 energy = 1.3841200740e+07
step 2 energy = 1.4185728788e+07
step 3 energy = 1.4616659685e+07
step 4 energy = 1.5092604878e+07
step 5 energy = 1.5590557984e+07
step 6 energy = 1.6109811701e+07
step 7 energy = 1.6639920746e+07
step 8 energy = 1.7184973984e+07
step 9 energy = 1.7736868058e+07
```

FIGURE 3 – Sortie de la version initiale.

```
linaarab0@DESKTOP-8M9P9CD:/mnt/c/Users/Microsoft/Desktop/UVSQ/AOC/TP1$ ./student 5000 10
Generating random input
Loading input
Computing 10 time steps
step 0 energy = 1.3813470141e+07
step 1 energy = 1.3841200740e+07
step 2 energy = 1.4185728788e+07
step 3 energy = 1.4616659685e+07
step 4 energy = 1.5092604878e+07
step 5 energy = 1.5590557984e+07
step 6 energy = 1.6109811701e+07
step 7 energy = 1.6639920746e+07
step 8 energy = 1.7184973984e+07
step 9 energy = 1.7736868058e+07
```

FIGURE 4 – Sortie de la version optimisée.

Je peux obtenir de petites différences si la réduction est vectorisée, car l'ordre des additions flottantes peut changer. Je vérifie que les valeurs restent cohérentes et que la tendance est identique.

9 Limites et pistes d'amélioration

- **Parallélisme multi-cœurs** : je force surtout SIMD avec `omp simd`, mais je peux aussi paralléliser la boucle externe sur i avec `#pragma omp parallel for`, en gardant une réduction sur E .
- **Tiling** : je peux bloquer la grille pour améliorer la réutilisation en cache si la taille des données et le stencil deviennent plus complexes.
- **Rapport compilateur** : je peux ajouter `-fopt-info-vec` avec GCC pour confirmer explicitement quelles boucles sont vectorisées.

10 Conclusion

En partant d'un code dominé par l'I/O ASCII et une organisation mémoire peu favorable, j'obtiens un speedup d'environ $30\times$ grâce à :

- la suppression du goulot lié à l'I/O,
- l'amélioration des accès mémoire (SoA, stride-1, fusion de boucles),
- l'alignement et `restrict` qui rendent la vectorisation SIMD plus efficace.

Les rapports MAQAO confirment la diminution du temps total et l'amélioration de l'efficacité mémoire.

Références

- [1] AOC, *Processor Architecture*, slides `lecture-1-v2.pdf`.
- [2] AOC, *Vectorization*, slides `vectorization.pdf`.